Flow, Controls und Feedback

Vom Key Screen zum klickbaren Hauptflow

Beschreibung

Das Zooh!-Projektteam hat in 6.1 jeden Key Screen als Route angelegt. Jetzt wird daraus ein Ablauf: Die Besucherin tippt sich durch den Hauptflow, füllt ein Formular aus und bekommt nach jeder Aktion eine Rückmeldung. Sie ergänzen Ihren Prototyp um Navigation, passende Controls und Feedback und prüfen den Flow zum Schluss in der Gruppe. Pflichtteile ca. 50 Minuten; Sie arbeiten in Ihrer Projektgruppe auf dem Schulnotebook.

Inhaltsverzeichnis

1. EINSTIEG UND THEORIE 2

1.1 AUSGANGSLAGE UND ZIELE 2
1.2 THEORIE: NAVIGATION MIT EXPO-ROUTER 2
1.3 THEORIE: CONTROLS, EINGABEN UND FEEDBACK 2

2. AUFGABE ABLAUF 6

2.1 EINFÜHRUNG 6
2.2 AUFGABE 1: HAUPTFLOW KLICKBAR 6
2.3 AUFGABE 2: FORMULAR MIT FELD 6
2.4 AUFGABE 3: ZÄHLER ODER SWITCH 6
2.5 AUFGABE 4: WALKTHROUGH IN DER GRUPPE 6
2.6 ZUSATZ FÜR SCHNELLE GRUPPEN (ADVANCED) 7

3. ERGEBNISPRÜFUNG UND ABGABE 7

3.1 CHECKLISTE 7
3.2 CHECKPOINT-FRAGEN 8

1. Einstieg und Theorie

1.1 Ausgangslage und Ziele

Der Klickprototyp aus 6.1 besteht aus einzelnen Screens, die nur der Startscreen verlinkt. Testpersonen brauchen aber einen durchgehenden Weg mit Weg zurück. Diese Aufgabe ist Iteration 1 Ihres Prototyps (40 % der Modulnote); die Tests in Block 7 laufen auf diesem Stand.
Nach dieser Aufgabe können Sie:

- den Hauptflow Ihrer User Journey als klickbare Navigation mit Weg zurück umsetzen (B2 «Benutzerschnittstelle entwerfen», Niveau advanced).
- zu einer Eingabe das passende Control wählen und ein Formular mit Beschriftung, Platzhalter und Fehlermeldung bauen (C1 «Controls, Widgets und Prototyp», Niveau intermediate).
- nach Aktionen Rückmeldung geben und Fehler am Feld verständlich erklären (B3 «Eingaben, Hilfe und Feedback», Niveau intermediate).

Aufgabe	Zeit	Produkt
1 Hauptflow klickbar	15 min	Screens verlinkt, Weg zurück
2 Formular mit Feld	12 min	Screen mit Beschriftung und Fehlertext
3 Zähler oder Switch	10 min	abgeleiteter Wert oder Rückmeldung
4 Walkthrough in der Gruppe	13 min	Screenshot pro Key Screen
Zusatz (advanced)	offen	Route `programm/[id]`

1.2 Theorie: Navigation mit expo-router

Jede Datei in `src/app/` ist eine Route; der `Stack` in `_layout.tsx` legt die Kopfzeile mit Zurück-Pfeil automatisch an. Vier Bausteine reichen für den Hauptflow:

Baustein	Einsatz
`<Link href="/tickets">`	Textlink zu einer Route
`router.push(...)`	Wechsel nach einer Aktion
`router.back()`	Zurück ohne Kopfzeile
`useLocalSearchParams()`	Wert der vorigen Seite lesen

Jeder Screen braucht einen Weg vor und zurück, sonst endet der Test in der Sackgasse.

Hinweis: Speichern Sie nur, was die Person eingibt (Anzahl), und leiten Sie das Total daraus ab. Ein eigener State fürs Total läuft auseinander.

1.3 Theorie: Controls, Eingaben und Feedback

Das Control richtet sich nach der Eingabe (Steimle & Wallach, S. 149–160 zur Nachbearbeitung, B3 laut Kompetenzmatrix):

Eingabe	Control	Merkmal
Anzahl 0 bis 10	`Zaehler` (– und +)	kein Tippen nötig
An oder aus	`Switch`	sofortige Wirkung
E-Mail, Name	`TextInput` mit Label	passende Tastatur
Aktion auslösen	`PrimaerButton`	grosse Tippfläche

Dazu die drei Regeln aus B3: Das **Label** bleibt sichtbar (der Platzhalter ist nur ein Beispiel und verschwindet beim Tippen). Die **Fehlermeldung** steht am Feld und nennt die Lösung («Bitte eine E-Mail-Adresse mit @ eingeben»). Nach jeder Aktion folgt **Feedback**: ein neuer Wert, ein Text oder ein deaktivierter Button.
ISO 9241-110 nennt sieben Grundsätze. Zwei davon prüfen Sie heute: **Selbstbeschreibungsfähigkeit** (jeder Screen sagt, wo man ist und was man tun kann) und **Fehlertoleranz** (ein Fehler kostet wenig Aufwand und wird verständlich erklärt).
Die drei Komponenten für den Prototyp. Zuerst der Button in `src/components/primaer-button.tsx`:

```
import { Pressable, StyleSheet, Text } from "react-native";
import { abstand, farben } from "@/theme";

type Props = { titel: string; onPress: () => void; deaktiviert?: boolean };

export function PrimaerButton(props: Props) {
  const { titel, onPress, deaktiviert = false } = props;
  return (
    <Pressable
      onPress={onPress}
      disabled={deaktiviert}
      accessibilityRole="button"
      style={[styles.button, deaktiviert ? styles.aus : null]}
    >
      <Text style={styles.text}>{titel}</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: { backgroundColor: farben.primaer, padding: abstand.m },
  aus: { backgroundColor: farben.rahmen },
  text: { color: farben.primaerText, fontSize: 16, textAlign: "center" },
});
```

Der Zähler in `src/components/zaehler.tsx` kennt seinen Wert nicht selbst; der Screen besitzt ihn:

```
import { Pressable, StyleSheet, Text, View } from "react-native";
import { abstand, farben } from "@/theme";

type Props = { label: string; wert: number; onChange: (neu: number) => void };

export function Zaehler({ label, wert, onChange }: Props) {
  const knopf = (zeichen: string, name: string, neu: number) => (
    <Pressable
      accessibilityRole="button"
      accessibilityLabel={`${label}: ${name}`}
      onPress={() => onChange(neu)}
    >
      <Text style={styles.knopf}>{zeichen}</Text>
    </Pressable>
  );
  return (
    <View style={styles.zeile}>
      <Text style={styles.text}>{label}</Text>
      {knopf("−", "eins weniger", Math.max(0, wert - 1))}
      <Text style={styles.text}>{wert}</Text>
      {knopf("+", "eins mehr", wert + 1)}
    </View>
```

```
  );
}

const styles = StyleSheet.create({
  zeile: { flexDirection: "row", alignItems: "center", gap: abstand.m },
  text: { fontSize: 16, color: farben.text },
  knopf: { fontSize: 28, color: farben.text, padding: abstand.s },
});
```

Das Eingabefeld in `src/components/feld.tsx` vereint Label, Platzhalter und Fehlertext:

```
import { StyleSheet, Text, TextInput, View } from "react-native";
import type { TextInputProps } from "react-native";
import { abstand, farben } from "@/theme";

type Props = TextInputProps & { label: string; fehler?: string };

export function Feld({ label, fehler, ...rest }: Props) {
  return (
    <View style={styles.gruppe}>
      <Text style={styles.label}>{label}</Text>
      <TextInput
        style={styles.eingabe}
        placeholderTextColor={farben.textSekundaer}
        accessibilityLabel={label}
        {...rest}
      />
      {fehler ? <Text style={styles.fehler}>{fehler}</Text> : null}
    </View>
  );
}

const styles = StyleSheet.create({
  gruppe: { gap: abstand.s },
  label: { fontSize: 14, color: farben.text },
  eingabe: { borderWidth: 1, borderColor: farben.rahmen, padding: 12 },
  fehler: { color: farben.fehler, fontSize: 14 },
});
```

Achtung: Schreiben Sie nie `{anzahl && <Text>…</Text>}` mit einer Zahl. Bei 0 erscheint die Null als Text auf dem Screen (in React Native sogar mit Fehler). Verwenden Sie `{anzahl > 0 ? <Text>…</Text> : null}`.

Der Weg zur nächsten Seite mit einem Wert. Der Screen `tickets.tsx` schickt das Total als Parameter (Parameter sind immer Text, daher `String`):

```
import { useRouter } from "expo-router";
import { useState } from "react";
import { ScrollView, StyleSheet, Text } from "react-native";
import { PrimaerButton } from "@/components/primaer-button";
import { Zaehler } from "@/components/zaehler";
import { abstand, farben } from "@/theme";

export default function Tickets() {
  const router = useRouter();
  const [erwachsene, setErwachsene] = useState(1);
  const total = erwachsene * 30;

  return (
    <ScrollView
      contentInsetAdjustmentBehavior="automatic"
```

```
      contentContainerStyle={styles.inhalt}
    >
      <Zaehler label="Erwachsene" wert={erwachsene} onChange={setErwachsene} />
      <Text style={styles.total}>Total: CHF {total}</Text>
      <PrimaerButton
        titel="Weiter zur Zahlung"
        deaktiviert={total === 0}
        onPress={() =>
          router.push({
            pathname: "/zahlung",
            params: { total: String(total) },
          })
        }
      />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  inhalt: { padding: abstand.m, gap: abstand.l },
  total: { fontSize: 20, color: farben.text },
});
```

Und die Zielseite `zahlung.tsx` liest den Wert wieder aus:

```
import { useLocalSearchParams } from "expo-router";
import { useState } from "react";
import { StyleSheet, Text, View } from "react-native";
import { Feld } from "@/components/feld";
import { abstand, farben } from "@/theme";

export default function Zahlung() {
  const { total } = useLocalSearchParams<{ total: string }>();
  const [email, setEmail] = useState("");
  const ungueltig = email.length > 0 && !email.includes("@");
  const fehler = ungueltig ? "Bitte eine E-Mail mit @ eingeben." : undefined;

  return (
    <View style={styles.inhalt}>
      <Text style={styles.text}>Zu bezahlen: CHF {total}</Text>
      <Feld
        label="E-Mail für das Ticket"
        placeholder="name@beispiel.ch"
        keyboardType="email-address"
        value={email}
        onChangeText={setEmail}
        fehler={fehler}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  inhalt: { padding: abstand.m, gap: abstand.l },
  text: { fontSize: 16, color: farben.text },
});
```

2. Aufgabe Ablauf

2.1 Einführung

Sie arbeiten in Ihrer festen Projektgruppe (3–4 Personen) an Ihrem Projekt aus 6.1. Pflichtteile: **50 Minuten** (15 + 12 + 10 + 13). Die Beispiele oben nennen Tickets und Zahlung; übertragen Sie sie auf **Ihren** Hauptflow aus 5.1.

Rolle	Aufgabe
Build-Verantwortliche(r)	Server, Fehlermeldungen, Zusammenführen
Screens (zwei Personen)	Aufgaben 1 bis 3 auf je einem Screen
Review (bei 4 Personen)	prüft Flow und Rückmeldungen

2.2 Aufgabe 1: Hauptflow klickbar

Zeit: ca. 15 min, Gruppe

1. Legen Sie die drei Komponenten aus der Theorie in `src/components/` an.
2. Schreiben Sie Ihren Hauptflow aus 5.1 als Kette auf (z. B. Start, Tickets, Zahlung, Mein Ticket).
3. Verbinden Sie jeden Screen mit dem nächsten: per `PrimaerButton` und `router.push`, am Startscreen per `Link`.
4. Prüfen Sie auf jedem Screen den Weg zurück: Kopfzeile des `Stack` oder ein Button mit `router.back()`.

→ Prüfen Sie: Kommt man vom ersten bis zum letzten Screen und wieder zurück, ohne in einer Sackgasse zu landen?

→ Produkt: Hauptflow vom Startscreen bis zum Ziel klickbar, jeder Screen mit Weg zurück

2.3 Aufgabe 2: Formular mit Feld

Zeit: ca. 12 min, Screens-Personen

1. Wählen Sie den Screen mit einer Eingabe (E-Mail, Name oder Suchbegriff) und setzen Sie `Feld` ein.
2. Vergeben Sie sichtbares **Label**, **Platzhalter** als Beispiel und die passende Tastatur über `keyboardType`.
3. Zeigen Sie bei ungültiger Eingabe einen **Fehlertext am Feld**, der die Lösung nennt.
4. Der Button erscheint erst, wenn die Eingabe gültig ist, oder bleibt bis dahin deaktiviert.

→ Produkt: Formular-Screen mit Label, Platzhalter, `keyboardType` und Fehlertext

2.4 Aufgabe 3: Zähler oder Switch

Zeit: ca. 10 min, Screens-Personen

1. Wählen Sie die Variante, die zu Ihrem Flow passt: **Zähler** mit abgeleitetem Total (wie im Beispiel `tickets.tsx`) oder **Switch** mit Feedbacktext («Erinnerung aktiv» / «Keine Erinnerung»).
2. Speichern Sie nur den eingegebenen Wert im State. Total oder Text leiten Sie bei jedem Rendern daraus ab.
3. Geben Sie das Total oder den Wert mit `router.push` und `params` an den nächsten Screen weiter und zeigen Sie es dort an.

→ Produkt: Screen mit Zähler oder Switch, Rückmeldung sofort sichtbar, Wert auf der Folgeseite

2.5 Aufgabe 4: Walkthrough in der Gruppe

Zeit: ca. 13 min, Gruppe

1. Eine Person spielt die Persona und löst eine Aufgabe aus Ihrem Hauptflow («Kaufen Sie zwei Tickets»); die anderen schweigen und notieren.

2. Bei jedem Screen fragen Sie: Weiss die Person, was sie tun kann? Sieht sie, dass ihre Aktion gewirkt hat? Findet sie zurück?
3. Notieren Sie pro Screen höchstens einen Befund in einem Satz, ohne Namen.
4. Machen Sie pro Key Screen einen Screenshot (Smartphone oder Browser) und legen Sie ihn mit den Befunden im Board ab.

→ Produkt: ein Screenshot pro Key Screen mit höchstens einem Befund (Selbstbeschreibungsfähigkeit, Fehlertoleranz)

2.6 Zusatz für schnelle Gruppen (advanced)

Bauen Sie eine **dynamische Route** `src/app/programm/[id].tsx`: Die Programmpunkte liegen in `src/daten/programm.ts`, der Screen liest die `id` aus der Route und setzt den Titel mit `Stack.Screen options={{ title }}`.

```
export type Programmpunkt = { id: string; titel: string; zeit: string };

export const programm: Programmpunkt[] = [
  { id: "pinguine", titel: "Pinguin-Fütterung", zeit: "13:30" },
  { id: "elefanten", titel: "Elefantenbad", zeit: "14:15" },
];

export function findeProgrammpunkt(id: string | undefined) {
  return programm.find((p) => p.id === id);
}
```

Der Screen dazu:

```
import { Stack, useLocalSearchParams } from "expo-router";
import { StyleSheet, Text, View } from "react-native";
import { findeProgrammpunkt } from "@/daten/programm";
import { abstand, farben } from "@/theme";

export default function Programmpunkt() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const punkt = findeProgrammpunkt(id);

  if (!punkt) return <Text style={styles.text}>Nicht gefunden.</Text>;

  return (
    <View style={styles.inhalt}>
      <Stack.Screen options={{ title: punkt.titel }} />
      <Text style={styles.text}>{punkt.zeit} Uhr</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  inhalt: { padding: abstand.m },
  text: { fontSize: 16, color: farben.text },
});
```

Verlinkt wird mit `href={{ pathname: "/programm/[id]", params: { id: "pinguine" } }}`.

3. Ergebnisprüfung und Abgabe

3.1 Checkliste

Prüfen Sie vor der Abgabe:

- Der Hauptflow ist vom Startscreen bis zum Ziel klickbar, jeder Screen hat einen Weg zurück.

- Mindestens ein Formular nutzt `Feld` mit sichtbarem Label, Platzhalter, `keyboardType` und Fehlertext am Feld.
- Ein Zähler (Total abgeleitet, nicht als State) oder ein Switch gibt sofort Rückmeldung.
- Jeder `PrimaerButton` hat `accessibilityRole="button"`; Text steht nur in `<Text>`.
- Pro Key Screen liegt ein Screenshot im ZIP oder Board; keine Namen von Mitlernenden oder Testpersonen.

3.2 Checkpoint-Fragen

Beantworten Sie die Fragen zu zweit; die Lehrperson fragt stichprobenartig.

1. Wann verwenden Sie `Link`, wann `router.push`, und wie kommt ein Wert auf die nächste Seite?
2. Warum ist das Total kein State, und was wäre die Folge?
3. Wozu dient das Label, wenn das Feld schon einen Platzhalter hat?
4. Welche zwei Grundsätze der ISO 9241-110 haben Sie im Walkthrough geprüft?

→ Abgabe: ZIP des Projekts **ohne** `node_modules` (oder Link zum Git-Repository) mit je einem Screenshot pro Key Screen in der Teams-Aufgabe «6.2 Flow und Controls» bis So 08.11.2026. Bewertet werden Flow, Controls und Feedback, nicht Code-Qualität oder visuelles Design. Eine Abgabe pro Gruppe; keine Namen von Testpersonen. Die Lösungen werden nach der Abgabe freigeschaltet.